STUDY CASE

Sistem Informasi Kemitraan Reseller pada Perusahaan Footwear Harmon Corp

Nama Kelompok :
Valentino Putra Prasetyo (22031554002)
Siti Sirotul Azhar (22031554026)
Joevita Salsabila Fitrianova (22031554031)

PROGRAM STUDI SAINS DATA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS NEGERI SURABAYA
2024

DAFTAR ISI

Analisis Proses Bisnis Perusahaan Footware Harmon.Corp....3
A. Deskripsi Umum....3
B. Alur Proses Bisnis....3
C. Aktor Utama....3
D. Kebutuhan Sistem....3
Sistem Informasi Kemitraan Reseller pada Perusahaan Footware Harmon Corp menggunakan Metode Extreme Programming (XP)....4
A. Perencanaan (Planning)....5
B. Perancangan Desain (Design)....6
C. Pengkodean/Iterasi (Coding/Iteration)....14
D. Pengujian (Testing)....22
Implementasi Design Pattern....24
A. Chain of Responsibility....24
B. Template Method....27
C. Strategy....31
REFERENSI....32

Analisis Proses Bisnis Perusahaan Footware Harmon.Corp

A. Deskripsi Umum

Sistem Penyedia Reselling Produk merupakan sistem yang dirancang untuk mendukung proses reselling produk alas kaki pada perusahaan Harmon.Corp, yang memungkinkan reseller untuk membeli produk melalui platform, lalu menjualnya kembali dan melaporkan hasil penjualan ke perusahaan melalui sistem. Dengan adanya sistem ini, proses bisnis antara perusahaan dan reseller dapat berjalan lebih terorganisir dan efisien.

B. Alur Proses Bisnis

Berikut merupakan alur proses bisnis Penyedia Reselling Produk :

1. Register Reseller : Calon reseller mendaftar melalui platform dengan mengisi data yang diperlukan, kemudian akan diverifikasi oleh admin.
2. Verifikasi Data : Admin memeriksa kelengkapan dan keabsahan atau kebenaran data. Jika data sesuai, maka akun reseller akan diaktifkan.
3. Pemesanan produk : Reseller memesan produk melalui katalog platform, memilih jumlah produk, melakukan pembayaran, dan menerima konfirmasi pemesanan.
4. Pengelolaan Stok : Sistem otomatis memperbarui stok setelah transaksi, sementara admin memantau stok dan menambah jika diperlukan.
5. Pelaporan dan Penjualan : Reseller melaporkan hasil penjualan harian melalui platform dengan mencatat data produk, jumlah, dan pendapatan.

Proses ini memastikan setiap langkah reselling terintegrasi dan mendukung operasional reseller serta perusahaan.

C. Aktor Utama

1. Reseller : Individu atau pihak yang menjual kembali produk perusahaan. Mendaftar, memesan produk, dan menjual produk ke pelanggan.
2. Admin : Pengelola sistem yang bertanggung jawab untuk verifikasi data, pemantauan stok, dan pengelolaan laporan.
3. Sistem : Platform yang mendukung proses bisnis reselling. Menampilkan katalog produk, proses pemesanan dan pembayaran.

D. Kebutuhan Sistem

Fungsional :

1. Registrasi dan login untuk reseller
   Sistem menyediakan form registrasi untuk reseller, memvalidasi data secara otomatis dan memungkinkan admin untuk memverifikasi keaslian data.
2. Pemesanan produk

Sistem memungkinkan reseller untuk melihat katalog, memilih produk, dan menyelesaikan pembayaran, serta memperbarui stok secara otomatis setelah transaksi.

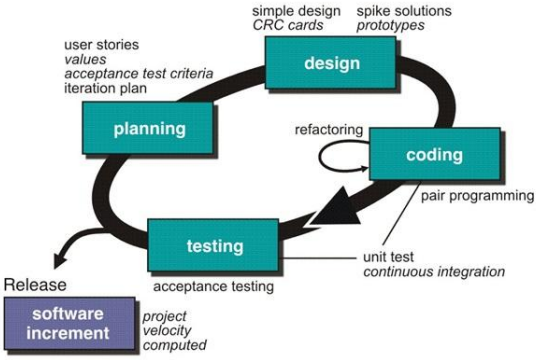
3. Pengelolaan stok
   Sistem mencatat perubahan stok secara real-time dan memberikan peringatan stok rendah kepada admin.
4. Pencatatan laporan penjualan
   Sistem menyediakan fitur untuk mengunggah laporan penjualan harian bagi reseller dan dapat diakses oleh admin.
5. Notifikasi
   Sistem mengirimkan notifikasi status pendaftaran, konfirmasi pemesanan, dan laporan stok kepada pengguna terkait.

Non-Fungsional :

1. Sistem melindungi keamanan data reseller dan transaksi dengan enkripsi serta memiliki otentikasi pengguna yang aman
2. Sistem dapat diakes selama 24 jam dengan waktu downtime minimal
3. Sistem memproses registrasi, pembayaran, dan pembaruan stok secara cepat
4. Sistem dapat menangani peningkatan jumlah reseller dan transaksi tanpa penurunan kinerja
5. Sistem memiliki antarmuka yang mudah digunakan oleh reseller dan admin

Sistem Informasi Kemitraan Reseller pada Perusahaan Footware Harmon Corp menggunakan Metode Extreme Programming (XP)

Metodologi Extreme Programming (XP) digunakan untuk membangun sistem dalam studi kasus ini. Metode Extreme Programming (XP) ini bersifat ringan, fleksibel dan berisiko rendah dengan kemampuan untuk mengelola requirement yang tidak jelas atau berubah dengan cepat, sehingga cocok untuk digunakan dalam tim dengan skala kecil dan menengah. Pada metode Extreme Programming (XP) ini terdapat fitur yang membedakan dengan metode-metode lain yaitu pair programming. Untuk mencegah kesalahan dan kesalahan pengkodean, dua programmer berkolaborasi pada satu komputer selama proses pair programming. Metode ini berfokus pada aspek teknik dibandingkan dengan manajemen proyek. Extreme Programming (XP) sangat menekankan kepuasan pelanggan dengan menggunakan feedback pada saat pengembangan perangkat lunak tersebut sedang berjalan, sehingga dapat membantu meminimalisir kegagalan proyek dan menghasilkan perangkat lunak sesuai dari kebutuhan pelanggan [1].



Gambar 1. Tahapan Metode Extreme Programming (XP)

Berikut merupakan tahapan-tahapan yang terdapat pada metode pengembangan sistem Extreme Programming (XP), yakni :

A. Perencanaan (Planning)

1). Identifikasi Masalah

Berdasarkan studi kasus awal, masalah utama yang dihadapi Harmon Corp. terletak pada kebutuhan akan sistem yang dapat mendukung efisiensi dan integrasi proses bisnis dalam program reselling produk. Saat ini, proses registrasi dan verifikasi mitra reseller belum terkelola secara terpusat, sehingga berpotensi menyebabkan kesalahan atau keterlambatan dalam onboarding mitra. Selain itu, tidak adanya sistem terintegrasi untuk pemesanan, pencatatan laporan penjualan, dan pengelolaan stok menciptakan risiko ketidaksesuaian data, seperti stok yang tidak akurat atau laporan yang tidak lengkap. Harmon Corp. juga memerlukan dukungan digital untuk mempermudah pengelolaan mitra, meningkatkan visibilitas produk melalui iklan, serta memastikan peningkatan volume penjualan tanpa membebani operasional. Oleh karena itu, diperlukan solusi perangkat lunak yang dapat mengelola seluruh proses secara real-time, aman, dan scalable untuk mendukung pertumbuhan bisnis perusahaan.

2). Analisis Kebutuhan Sistem

Analisis kebutuhaan yaitu menganalisa jenis kebutuhan berisi proses-proses apa saja yang nantinya dilakukan oleh sistem. Kebutuhaan fungsional berisi informasi apa saja yang harus ada dan dihasilkan oleh sistem [2]. Untuk mengatasi masalah di atas, Harmon Corp. membutuhkan sistem terintegrasi yang mampu mendukung program reselling secara efisien dan scalable. Sistem ini harus menyediakan fitur registrasi dan verifikasi mitra reseller dengan validasi data otomatis untuk memastikan proses onboarding berjalan cepat dan akurat. Selain itu, sistem perlu mendukung pemesanan produk oleh reseller dengan katalog yang mudah diakses, proses pembayaran yang aman, dan pembaruan stok secara otomatis. Fitur pencatatan laporan harian penjualan juga penting untuk memantau performa mitra dan menyediakan data yang dapat diakses oleh admin. Pengelolaan stok harus dilakukan secara real-time dengan notifikasi stok rendah untuk mencegah kekurangan produk.

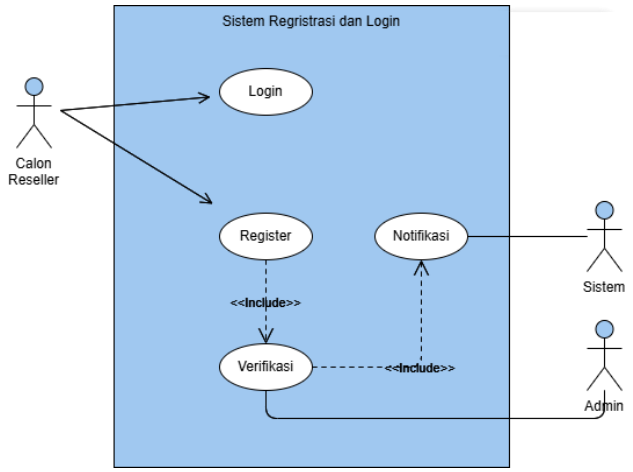
Secara non-fungsional, sistem harus aman dengan enkripsi data dan otentikasi pengguna, mampu beroperasi 24 jam dengan downtime minimal, serta dirancang agar

mudah digunakan oleh reseller dan admin. Solusi ini diharapkan mampu mendukung pertumbuhan bisnis perusahaan dengan meningkatkan efisiensi operasional dan memperluas pasar tanpa menambah biaya yang signifikan.

B. Perancangan Desain (Design)

1). Use Case Diagram

➢ Use Case Sistem Registrasi dan Login

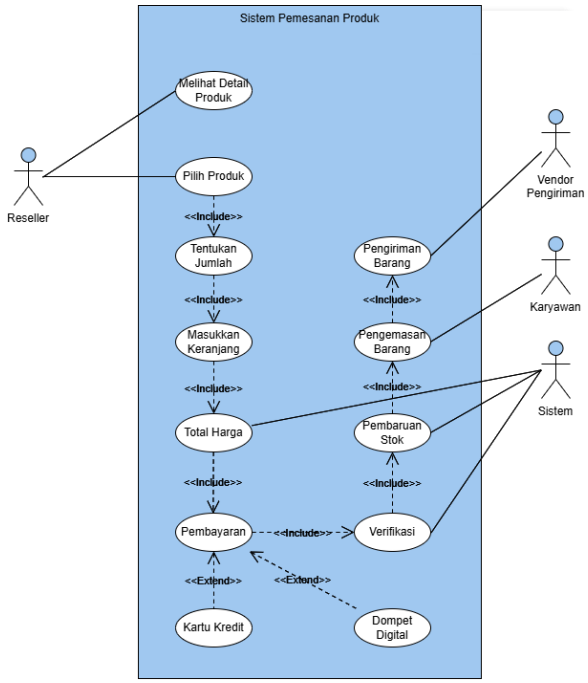


Gambar 1. Use Case Sistem Registrasi dan Login

- Aktor : reseller, admin, dan sistem
- Tujuan : mendaftarkan reseller baru pada platform reselling perusahaan, dan reseller dapat masuk ke sistem
- Alur Kerja :
    1. Reseller mengakses form registrasi melalui platform
    2. Reseller mengisi data registrasi yang diminta
    3. Sistem mevalidasi format data secara otomatis
    4. Data dikirim ke admin untuk verifikasi
    5. Admin memverifikasi data dan memberikan keputusan
    6. Sistem mengirimkan notifikasi kepada reseller mengenai status pendaftaran
    7. Reseller memasukkan data username untuk masuk ke akun
    8. Sistem memvalidasi kredensial
    9. Sistem memberikan akses jika kredensial valid
- Kondisi Awal : reseller belum terdaftar di platform dan mengakses formulir registrasi dan belum bisa masuk ke sistem
- Kondisi Akhir : reseller terdaftar dengan status aktif jika disetujui atau diminta untuk memperbaiki data jika ditolak dan reseller bisa masuk ke sistem
- Skenario Alternatif :
    1. Jika data tidak valid, sistem akan memberikan pemberitahuan kesalahan atau meminta perbaikan

2. Jika pendaftaran ditolak oleh admin, reseller diminta untuk memperbaiki data yang kurang lengkap atau tidak valid.
3. Jika kredensial tidak valid, sistem menampilkan pesan kesalahan

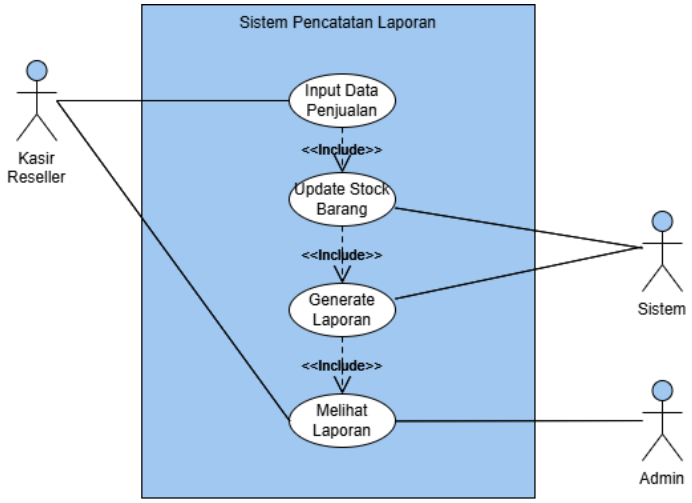
➢ Use Case Sistem Pemesanan Produk



Gambar 2. Use Case Sistem Pemesanan Produk

- Aktor : Reseller, sistem
- Tujuan : melakukan pemesanan produk untuk dijual kembali kepada pelanggan
- Alur Kerja :
    1. Reseller login ke akun dan membuka katalog produk
    2. Reseller memilih produk yang diinginkan
    3. Reseller menentukan jumlah produk yang akan dipesan
    4. Sistem menghitung total biaya dan membuat invoice
    5. Reseller memilih metode pembayaran
    6. Reseller melakukan pembayaran
    7. Sistem memvalidasi pembayaran
    8. Sistem memperbarui stok produk
    9. Sistem mengirimkan konfirmasi pemesanan kepada reseller
- Kondisi Awal : reseller sudah login ke akun dan mengakses katalog produk
- Kondisi Akhir : produk berhasil dipesan dan transaksi tercatat dengan stok yang diperbarui
- Skenario Alternatif :
    1. Jika pembayaran gagal, reseller diberitahu untuk mencoba metode pembayaran lain
    2. Jika stok tidak mencukupi, reseller diberitahu tentang ketersediaan produk yang kurang

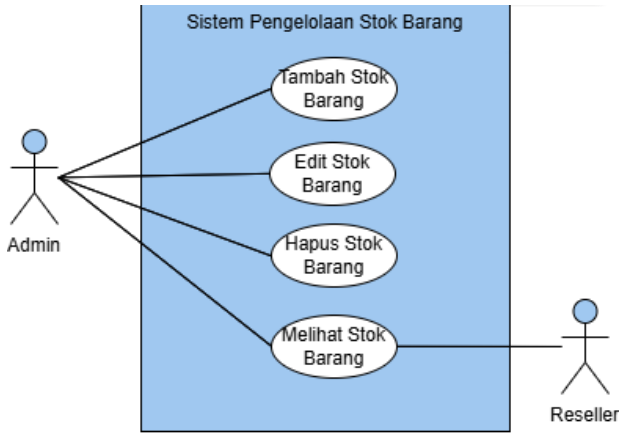
➢ Use Case Sistem Pencatatan Laporan Harian



Gambar 3. Use Case Sistem Pencatatan Laporan

- Aktor : reseller, sistem
- Tujuan : mencatat hasil penjualan harian untuk memantau performa dan melaporkan data ke perusahaan.
- Alur Kerja :
    1. Reseller login dan membuka halaman laporan penjualan
    2. Reseller mengisi data penjualan yang meliputi produk yang terjual, jumlah, harga jual, tanggal transaksi, dan total pendapatan
    3. Sistem memvalidasi format data yang dimasukkan
    4. Sistem menyimpan laporan ke database
    5. Sistem menghasilkan laporan harian yang dapat diakses oleh admin
- Kondisi Awal : reseller login dan mengakses halaman laporan penjualan
- Kondisi Akhir : laporan penjualan harian disimpan dan tersedia untuk dianalisis
- Skenario Alternatif :
    1. Jika data yang dimasukkan tidak valid, sistem memberi pemberitahuan untuk memperbaiki format data
    2. Jika laporan tidak dapat disimpan, sistem memberikan pesan eror

➢ Use Case Sistem Pengelolaan Stok



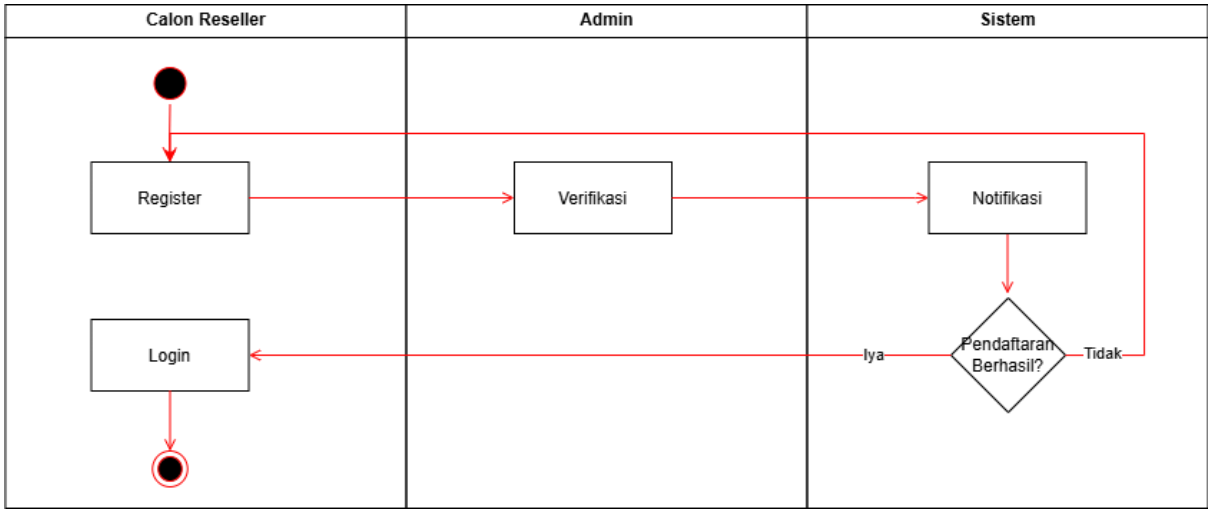
Gambar 4. Use Case Sistem Pencatatan Laporan

- Aktor : Admin, sistem

- Tujuan : memantau dan mengelola stok produk untuk memastikan ketersediaan produk bagi reseller
- Alur Kerja :
    1. Sistem memperbaiki stok setelah transaksi pemesanan dilakukan
    2. Admin memantau laporan stok melalui dashboard
    3. Sistem memberikan peringatan jika stok rendah
    4. Admin menambah stok pruduk
    5. Admin memperbarui data stok di sistem setelah stok baru diterima
    6. Sistem mencatat setiap perubahan pada stok
- Kondisi Awal : Stok tersedia dan dapat dipantau oleh admin
- Kondisi Akhir : Stok diperbarui dan tercatat dalam sistem, ketersediaan produk tetap optimal
- Skenario Alternatif :
    1. Jika stok tidak dapat diperbarui, admin menerima pemberitahuan untuk memperbaiki atau memverifikasi data stok
    2. Jika stok habis, admin diberitahu dan dapat menambah stok dari sumber lain

2). Activity Swimlane

➢ Swimlane Sistem Registrasi dan Login



Gambar 5. Swimlane Sistem Registrasi dan Login

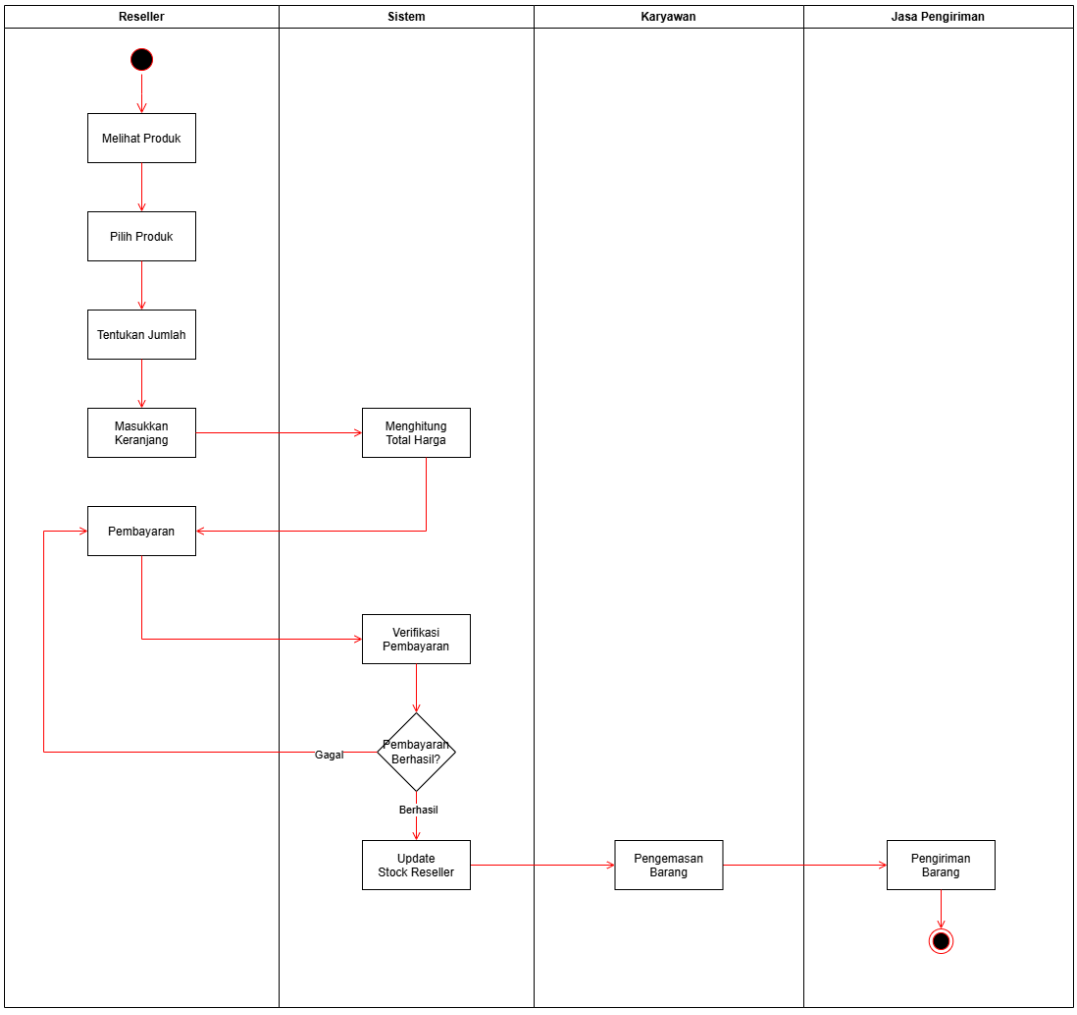
Berikut penjelasan alur activity swimlane pada sistem registrasi dan login :

1. Registrasi Reseller (Calon Reseller)
   Reseller memulai proses dengan mengisi formulir registrasi yang berisi informasi penting seperti nama, alamat email, dan data lainnya. Aktivitas ini terjadi di kolom Reseller. Setelah data dimasukkan, reseller mengirimkan formulir untuk diproses verifikasi lebih lanjut oleh admin.
2. Verifikasi Data (Admin)
   Setelah sistem melakukan validasi awal, data registrasi dikirimkan ke admin untuk diverifikasi lebih lanjut. Admin akan memeriksa kelengkapan dan kebenaran informasi yang diberikan oleh reseller. Jika ada kekurangan atau

ketidaksesuaian, admin dapat menolak pendaftaran atau meminta reseller untuk memperbaiki data.

3. Notifikasi Status Pendaftaran (Sistem)
   Berdasarkan keputusan admin, sistem mengirimkan notifikasi kepada reseller mengenai status pendaftaran mereka. Jika pendaftaran diterima, reseller dapat melanjutkan ke tahap login. Jika pendaftaran ditolak, reseller akan diberi tahu untuk memperbaiki data mereka dan kembali ke tahap register.
4. Pendaftaran Berhasil (Calon Reseller)
   Jika pendaftaran disetujui oleh admin, reseller dapat melanjutkan untuk melakukan login menggunakan kredensial yang sudah didaftarkan sebelumnya. Proses ini dilanjutkan ke kolom Reseller yang mengakses sistem menggunakan username dan password yang telah dibuat.
5. Validasi Kredensial (Sistem)
   Sistem memvalidasi kredensial yang dimasukkan oleh reseller pada saat login. Jika kredensial valid, reseller diberi akses untuk menggunakan sistem dan melanjutkan aktivitas bisnisnya. Jika kredensial tidak valid, reseller akan diminta untuk memperbaiki atau mengulang proses login.

➢ Swimlane Sistem Pemesanan Produk

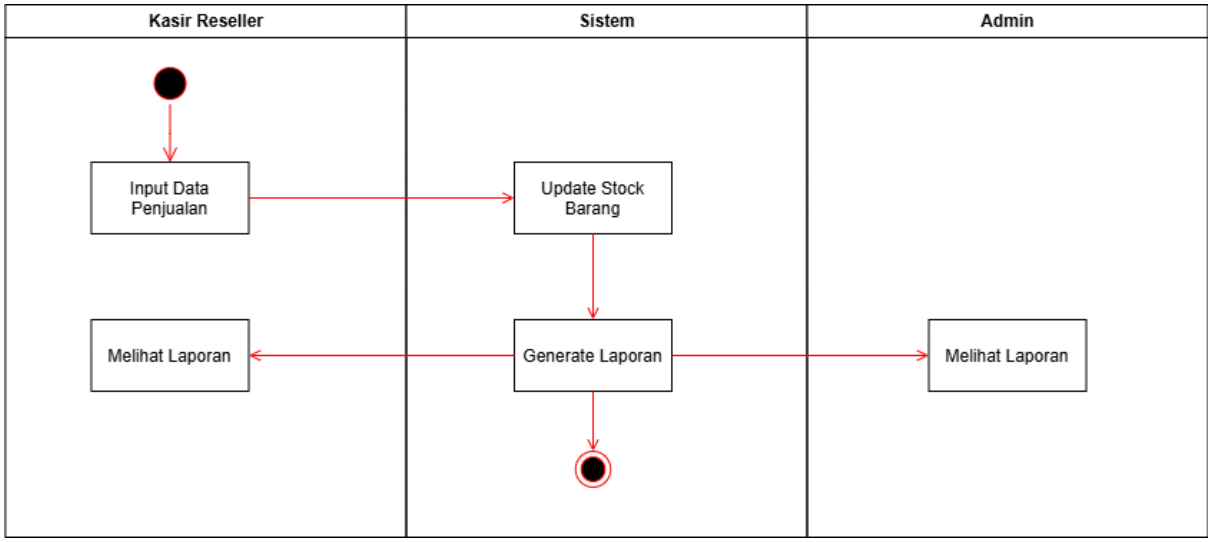


Gambar 6. Swimlane Sistem Pemesanan Produk

Berikut penjelasan alur activity swimlane pada sistem pemesanan produk :

1. Melihat Produk (Reseller)
   Proses dimulai ketika reseller membuka katalog produk yang tersedia. Pada tahap ini, reseller dapat menelusuri berbagai produk yang dijual dan memilih produk yang ingin mereka pesan.
2. Pilih Produk (Reseller)
   Setelah melihat katalog, reseller memilih produk yang akan dipesan. Produk ini dipilih sesuai dengan kebutuhan atau permintaan dari pelanggan.
3. Tentukan Jumlah (Reseller)
   Reseller menentukan jumlah unit produk yang akan dipesan, baik untuk memenuhi pesanan pelanggan atau untuk kebutuhan stok mereka sendiri.
4. Masukkan Keranjang (Reseller)
   Produk yang dipilih beserta jumlahnya dimasukkan ke dalam keranjang belanja. Ini adalah langkah persiapan sebelum transaksi dilanjutkan ke tahap pembayaran.
5. Menghitung Total Harga (Sistem)
   Setelah produk dan jumlahnya dimasukkan ke dalam keranjang, sistem secara otomatis menghitung total harga berdasarkan harga satuan setiap produk yang dipilih dan jumlah yang dimasukkan oleh reseller.
6. Pembayaran (Reseller)
   Reseller memilih metode pembayaran yang tersedia, seperti transfer bank, pembayaran digital, atau metode lainnya sesuai dengan opsi yang disediakan oleh sistem.
7. Verifikasi Pembayaran (Sistem)
   Sistem memverifikasi apakah pembayaran berhasil dilakukan. Sistem memeriksa validitas dan kecocokan pembayaran dengan data transaksi.
   - Pembayaran Berhasil**:** Jika pembayaran berhasil, proses berlanjut ke tahap berikutnya, yaitu pembaruan stok dan konfirmasi pemesanan.
   - Pembayaran Gagal**:** Jika pembayaran gagal, reseller akan diberitahu untuk mencoba metode pembayaran lain. Proses ini kembali ke tahap pembayaran untuk mencoba opsi pembayaran yang berbeda.
8. Update Stok Produk (Sistem)
   Setelah pembayaran berhasil, sistem mengurangi jumlah stok produk yang telah dipesan sesuai dengan jumlah yang dibeli oleh reseller. Pembaruan stok ini dilakukan secara otomatis untuk memastikan akurasi data persediaan.
9. Pengemasan Barang (Karyawan)
   Setelah stok dikurangi, karyawan akan menerima pesanan untuk dikemas. Produk yang dipesan disiapkan dan dibungkus untuk pengiriman sesuai dengan prosedur pengemasan yang ditentukan.
10. Pengiriman Barang (Jasa Pengiriman)
    Setelah barang selesai dikemas, karyawan menyerahkan paket kepada jasa pengiriman. Jasa pengiriman kemudian mengirimkan barang kepada reseller atau langsung ke pelanggan, tergantung pada perjanjian pengiriman.

➢ Swimlane Sistem Pencatatan Laporan Penjualan

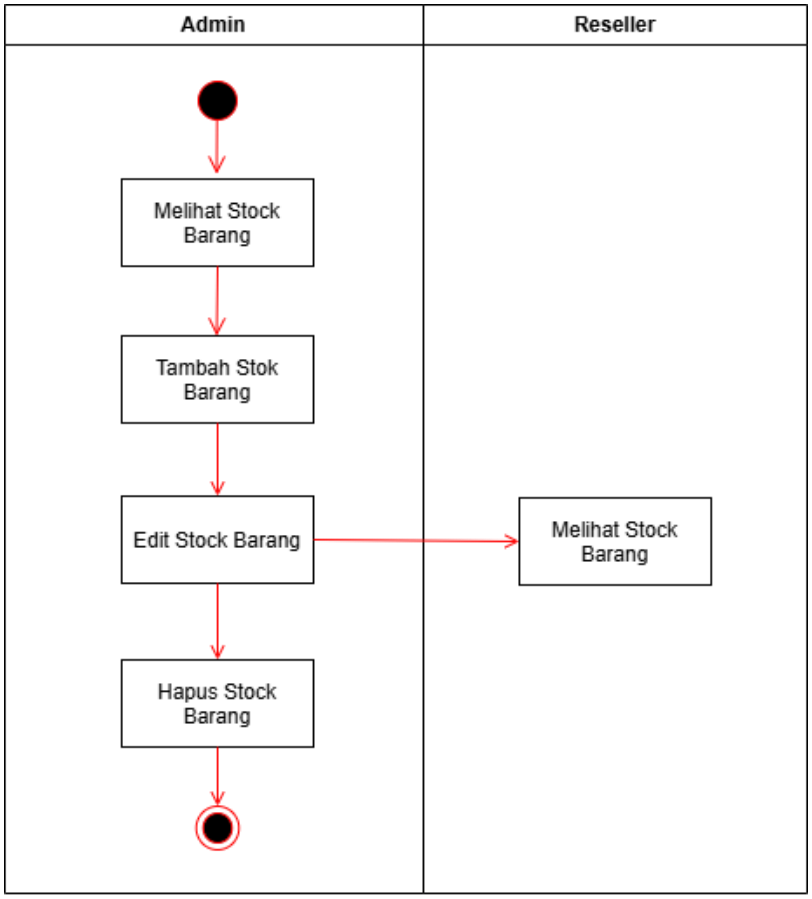


Gambar 7. Swimlane Sistem Pencatatan Laporan

Berikut penjelasan alur activity swimlane pada sistem pencatatan laporan :

1. Input Data Penjualan (Kasir Reseller)
   Kasir reseller memulai proses dengan menginputkan data penjualan yang mencakup informasi produk yang terjual, jumlah, harga jual, tanggal transaksi, dan total pendapatan. Proses ini merupakan langkah awal yang penting karena data yang dimasukkan akan menjadi dasar untuk laporan harian.
2. Update Stok Barang (Sistem)
   Setelah data penjualan divalidasi, sistem akan memperbarui stok barang secara otomatis berdasarkan data penjualan yang telah diterima. Ini memastikan bahwa jumlah stok barang yang tersedia di sistem selalu tercatat dengan akurat.
3. Generate Laporan (Sistem)
   Sistem secara otomatis menghasilkan laporan harian berdasarkan data penjualan yang telah diproses. Laporan ini mencakup rincian transaksi dan total pendapatan, yang dapat digunakan oleh kasir reseller dan admin untuk melacak performa penjualan.
4. Melihat Laporan (Kasir Reseller/Admin)
   Setelah laporan harian berhasil dihasilkan, baik kasir reseller maupun admin dapat mengakses laporan tersebut. Kasir reseller mungkin memerlukan laporan untuk keperluan pencatatan pribadi atau untuk memberikan informasi lebih lanjut kepada pelanggan, sementara admin memantau laporan untuk analisis lebih mendalam mengenai performa penjualan dan pengelolaan stok.

➢ Swimlane Sistem Pengelolaan Stok



Gambar 8. Swimlane Sistem Pengelolaan Stok

Berikut penjelasan alur activity swimlane pada sistem pengelolaan stok :

1. Melihat Stok Barang (Admin)
   Admin memiliki akses penuh terhadap status stok barang di seluruh sistem. Aktivitas ini memungkinkan admin untuk memantau ketersediaan produk secara keseluruhan, sehingga mereka dapat mengetahui jumlah stok yang ada dan memutuskan tindakan lebih lanjut.
2. Tambah Stok Barang (Admin)
   Admin berperan dalam menambah jumlah stok barang ketika ada pengiriman produk baru atau restock. Admin dapat memasukkan data baru atau menyesuaikan stok yang sudah ada, memastikan bahwa jumlah stok yang tercatat selalu akurat.
3. Edit Stok Barang (Admin)
   Admin juga memiliki hak untuk mengedit data stok barang, misalnya untuk memperbarui harga atau deskripsi produk. Aktivitas ini memungkinkan admin untuk memastikan bahwa informasi yang tersedia dalam sistem selalu up-to-date dan sesuai dengan kondisi pasar.
4. Melihat Stok Barang (Reseller)
   Reseller hanya diberikan akses untuk melihat informasi stok barang yang tersedia. Ini penting agar reseller dapat mengetahui produk mana yang dapat mereka jual, serta ketersediaan stok sebelum melakukan pemesanan. Reseller

tidak dapat mengubah atau menambah stok barang, hanya dapat melihat status stok yang ada.

5. Hapus Stok Barang (Admin)
   Admin memiliki otoritas untuk menghapus data stok barang, misalnya ketika produk sudah tidak lagi dijual atau jika terjadi kesalahan dalam pencatatan stok. Aktivitas ini membantu admin dalam mengelola data produk yang sudah tidak relevan atau tidak lagi tersedia.

C. Pengkodean/Iterasi (Coding/Iteration)

Proses pengembangan sistem reselling untuk Harmon Corp. mengikuti metode XP dengan iterasi kecil dan kolaborasi erat antara pengembang dan pengguna. Berikut adalah penjelasan tahapan pengkodean berdasarkan fitur-fitur yang telah dijelaskan sebelumnya :

- Iterasi 1 Registrasi dan Login Reseller

  Tahap ini dimulai dengan pengembangan fitur registrasi dan login. Fitur ini melibatkan pembuatan formulir pendaftaran yang memvalidasi data secara otomatis dan memungkinkan admin untuk memverifikasi keabsahan data reseller. Teknik pair programming digunakan untuk memastikan kode bebas dari kesalahan dan dapat langsung diintegrasikan. Output dari iterasi ini adalah fitur registrasi dan login yang siap digunakan oleh mitra reseller.

  Pseudocode Sistem Registrasi dan Login

  1. Tahap Registrasi :

```
BEGIN
    DISPLAY subheader "Formulir Registrasi" for Reseller
    DISPLAY info "Setelah registrasi, akun Anda harus diverifikasi
oleh admin sebelum bisa login."

    CREATE form registrasi for Reseller with inputs:
    - Email
    - Password
    - Konfirmasi Password
    - Nama Lengkap
    - Alamat
    - Nomor Kontak
```

  2. Tahap Verifikasi :

```
BEGIN
    IF user role is "admin":
    DISPLAY subheader "Verifikasi Reseller"
    TRY:
        READ data reseller dari file DATA_FILE into
dataframe (df)
```

FILTER reseller yang statusnya "unverified" into unverified_resellers

IF unverified_resellers is NOT empty:
DISPLAY list of unverified resellers (email, name, contact, status)

DISPLAY selectbox for admin to choose reseller by email

IF admin clicks "Verifikasi" button:
UPDATE the selected reseller's status to "verified" in the dataframe (df)
SAVE updated dataframe back to DATA_FILE
DISPLAY success message: "Reseller {selected_reseller} berhasil diverifikasi."
RERUN the page to reflect changes

ELSE:
DISPLAY info message: "Tidak ada reseller yang perlu diverifikasi."

EXCEPT FileNotFoundError:
DISPLAY error message: "File data reseller tidak ditemukan."
END

3. Tahap Notifikasi :

IF form is submitted:
INITIALIZE handler chain for data validation:
- EmailValidationHandler -> PasswordValidationHandler -> ConfirmPasswordHandler -> EmailExistsHandler -> SaveUserHandler

CREATE request_data dictionary containing:
- email (trimmed)
- password (trimmed)
- confirm_password (trimmed)
- name (trimmed)
- address (trimmed)
- contact (trimmed)
- role (Reseller, lowercase)
- status ("unverified" for Reseller)

EXECUTE handler chain to process request_data

IF result is "Registrasi berhasil.":
DISPLAY success message "Registrasi berhasil."
ELSE:
DISPLAY error message with result
END

4. Tahap Login :

BEGIN
IF action is "Login":
DISPLAY subheader "Login" for Reseller

GET email from input
GET password from input

IF "Login" button is clicked:
TRY:
READ user data from DATA_FILE into dataframe (df)
FIND user by matching email in the dataframe (df)
IF user is not found OR password does not match OR role is incorrect:
DISPLAY error message "Email atau password salah."
ELSE:
CREATE handler for verification status check (VerificationStatusHandler)
HANDLE verification status for the user (handle with email)

IF verification status indicates an issue:
DISPLAY error message based on result from handler
ELSE:
Set session state for successful login
STORE user data in session state
DISPLAY success message "Selamat datang, {user['name']}!"
RERUN the page to reflect logged-in state

EXCEPT FileNotFoundError:
DISPLAY error message "Database pengguna tidak ditemukan."
END

- Iterasi 2 Pemesanan dan Konfirmasi Pembayaran

Pada iterasi ini, pengembangan berfokus pada modul pemesanan produk yang mencakup pemilihan produk, pembayaran, dan proses pengemasan.

Sistem harus memungkinkan reseller untuk menentukan jumlah produk, menghasilkan invoice otomatis, dan menyediakan berbagai metode pembayaran. Proses pembayaran divalidasi secara otomatis, dan stok produk diperbarui setelah transaksi selesai. Setelah pembayaran, sistem akan mengintegrasikan proses pengemasan produk yang sesuai dengan pesanan dan memproses pengiriman ke reseller. Tahap ini memastikan pengalaman pemesanan yang aman dan efisien.

Pseudocode Sistem Pemesanan dan Konfirmasi Pembayaran

```
BEGIN
1. Tahap Melihat Produk
   stock_data = reseller.view_stock()  # Mendapatkan data stok
   DISPLAY "Data Stok untuk Reseller"
   DISPLAY stock_data  # Menampilkan data stok untuk reseller
2. Tahap Pilih Produk (Reseller)
   DISPLAY subheader "Pilih Produk untuk Dipesan"
   selected_product = reseller SELECT product from catalog
3. Tahap Menampilkan detail produk
   product_details = FIND product details in data where product name
   matches selected_product
   DISPLAY product_details
4. Tahap Menentukan Jumlah (Reseller)
      quantity = reseller INPUT quantity to order (min 1, max available stock)
5. Tahap Masukkan Keranjang (Reseller)
    FUNGSI add_to_cart (product_name, quantity, total_harga):
    PRICE = LOAD_DATA()["Price"] di MANA "Item" == product_name
    total_harga = PRICE * quantity
    TAMBAHKAN KE CART({"product_name": product_name, "quantity":
        quantity, "total_harga": total_harga})
6. Tahap Menghitung Total Harga (Sistem)
    FUNGSI get_cart_details():
        KEMBALIKAN KONVERSI_KE_TABEL(CART)
    FUNGSI calculate_total_amount():
        total_amount = 0
        UNTUK setiap item di CART:
        total_amount = total_amount + item["total_harga"]
        KEMBALIKAN total_amount
7. Tahap Pembayaran (Reseller) :
    payment_method = reseller SELECT payment method (Kartu Kredit or
        Dompet Digital)
        IF "Lakukan Pembayaran" button is clicked:
        IF payment_method is "Kartu Kredit":
                payment_method_instance = CreditCardPayment()
        ELSE IF payment_method is "Dompet Digital":
                payment_method_instance = DigitalWalletPayment()
```

CALL reseller.select_payment_method(payment_method_instance, total_amount)
DISPLAY success message: "Pembayaran sebesar {total_amount} berhasil dilakukan menggunakan {payment_method}."
Menampilkan pesanan yang telah dimasukkan reseller
DISPLAY "Pesanan Anda"
DISPLAY reseller.get_cart_details()
END

8. Tahap Verifikasi Pembayaran :

BEGIN
Verifikasi Pembayaran (Sistem)
total_amount = system CALL calculate_total_amount()
TRY:
READ existing transactions from file (transactions_file) into DataFrame (existing_data)
CONCATENATE new transaction data (from cart) with existing_data into new_data
SAVE new_data back to transactions_file (append new transaction data)
EXCEPT FileNotFoundError:
IF transactions_file does not exist, CREATE a new file with transaction data

PRINT "Transaksi berhasil dicatat dengan total pengeluaran {total_amount}"

Pembayaran Berhasil
IF payment is valid (successful):
SYSTEM: Proceed to update stock and confirm the order
DISPLAY success message: "Pembayaran berhasil! Pesanan sedang diproses."

Pembayaran Gagal
ELSE IF payment failed:
SYSTEM: Inform reseller to try another payment method
DISPLAY error message: "Pembayaran gagal. Silakan coba metode pembayaran lain."
RETURN to payment selection stage
END

9. Tahap Update Stok setelah Transaksi :

BEGIN
Update Stok Produk (Sistem)
LOAD stock data from "data_stock.csv" into stock_data

LOCATE the product in stock_data where product_name matches the selected product
DECREASE the quantity of the product by the purchased quantity

SAVE the updated stock data back to "data_stock.csv" file

PRINT "Stok untuk {product_name} berhasil diperbarui."
END

10. Tahap Pengemasan Barang :

```
Try:
    Load PRODUCT_FILE into dataframe df
    Filter rows where status == "unpacked" into undelivered_stock

    If undelivered_stock is not empty:
        Display "Barang yang perlu dikirim"
        Show undelivered_stock with selected columns

        Let user select an order index from undelivered_stock
        Show a button labeled "Sudah Dikemas"

        If button "Sudah Dikemas" is pressed:
            Update the selected row's status to "packed" in df
            Save updated df to PRODUCT_FILE
            Display success message for the selected index
            Rerun the interface

    Else:
        Display "Tidak ada barang yang perlu dikemas"

Except FileNotFoundError:
    Display "File data barang tidak ditemukan"
```

11. Tahap Pengiriman Barang :

```
If user['role'] == 'jasa pengiriman':
    Display "Data Pengiriman Barang Harmon Corp"

    Try:
        Load PRODUCT_FILE into dataframe df
        Filter rows where status == "packed" into packed_stock

        If packed_stock is not empty:
            Display "Barang yang perlu dikirim"
            Show packed_stock with selected columns

            Let user select an order index from packed_stock
```

Show a button labeled "Kirim"

If button "Kirim" is pressed:
Update the selected row's status to "delivered" in df
Save updated df to PRODUCT_FILE
Display success message for the selected index
Rerun the interface

Else:
Display "Tidak ada barang yang perlu dikirim"

Except FileNotFoundError:
Display "File data barang tidak ditemukan"

- Iterasi 3 Pencatatan Laporan Penjualan

Sistem pencatatan laporan penjualan dikembangkan untuk memungkinkan reseller mengunggah laporan penjualan harian mereka. Modul ini menyediakan formulir unggah yang terintegrasi dengan basis data, sehingga admin dapat memantau performa penjualan reseller secara real time. Data penjualan yang tercatat di sistem akan membantu perusahaan dalam menganalisis tren penjualan dan merencanakan strategi pemasaran yang lebih baik.

Pseudocode Sistem Pencatatan Laporan Penjualan

1. Tahap Input Data Penjualan

TAMPILKAN "Pelaporan Harian Penjualan"
PILIH_PRODUK(selected_product, user_products['product_name'])
DETAIL_PRODUK = FILTER(user_products, "product_name" == selected_product)

JIKA DETAIL_PRODUK ADA:
TAMPILKAN_INPUT(quantity,min=1,max=DETAIL_PRODUK['quantity'], step=1)
LAINNYA:
TAMPILKAN_ERROR("Produk tidak ditemukan.")

JIKA TOMBOL_DITEKAN("Masukkan ke Tabel Penjualan"):
total_harga = DETAIL_PRODUK['price'] * quantity
JIKA "sales" TIDAK ADA DI session_state:
INISIALISASI session_state.sales = []
TAMBAHKAN session_state.sales({"product_name": selected_product, "quantity": quantity, "total_penghasilan": total_harga})
TAMPILKAN_SUKSES("Produk berhasil ditambahkan ke Tabel Penjualan.”)

2. Tahap Update Stok Produk

TAMPILKAN "Tabel Penjualan"

JIKA session_state.sales TIDAK KOSONG:
TAMPILKAN_TABEL(session_state.sales)
JIKA TOMBOL_DITEKAN("Submit Laporan"):
JIKA session_state.sales TIDAK KOSONG:
UNTUK setiap item di session_state.sales:
product_name = item["product_name"]
qty_sold = item["quantity"]
df['quantity'] -= qty_sold DI MANA df['product_name'] == product_name
SIMPAN df KE PRODUCT_FILE
TAMPILKAN_SUKSES("Laporan berhasil disubmit dan stok telah diperbarui.")
RESET session_state.sales = []
RERUN_PAGE()
LAINNYA:
TAMPILKAN_WARNING("Tabel Penjualan kosong. Tambahkan produk terlebih dahulu.")

- Iterasi 4 Pengelolaan Stok dan Notifikasi

Modul pengelolaan stok dibangun untuk mencatat perubahan stok secara real time. Sistem akan memberikan peringatan kepada admin jika stok mendekati ambang batas minimum, memungkinkan pengisian ulang secara tepat waktu. Selain itu, sistem mengirimkan notifikasi kepada reseller terkait status pendaftaran, konfirmasi pemesanan, dan informasi stok produk. Integrasi ini memastikan pengelolaan stok yang efisien dan komunikasi yang lancar antara perusahaan dan reseller.

Pseudocode Sistem Pencatatan Laporan Penjualan

Tahap Menambahkan Produk :

FUNGSI add_stock(item, quantity, price):
DATA = LOAD_DATA()
NEW_DATA = BUAT_TABEL([item], [quantity], [price])
DATA = GABUNGKAN(DATA, NEW_DATA)
SAVE_DATA(DATA)
RETURN "Produk berhasil ditambahkan"

Tahap Mengedit Stok :

FUNGSI edit_stock(product_name, updated_quantity, updated_price):
DATA = LOAD_DATA()
JIKA DATA["Item"] == product_name:
DATA["Quantity"] = updated_quantity
DATA["Price"] = updated_price
AKHIR JIKA
SAVE_DATA(DATA)
RETURN "Produk berhasil diperbarui"

Tahap Menghapus Stok :

FUNGSI delete_stock(product_name):

```
DATA = LOAD_DATA()
DATA = FILTER(DATA, "Item" != product_name)
SAVE_DATA(DATA)
RETURN "Produk berhasil dihapus"
```

- Refactoring di Setiap Iterasi

  Selama proses iterasi, kode yang sudah ditulis akan diperbaiki secara berkala untuk meningkatkan efisiensi dan kualitas tanpa mengubah fungsionalitas. Langkah ini membantu menjaga struktur kodel tetap bersih dan mendukung integrasi fitur baru secara berkelanjutan.

D. Pengujian (Testing)

Modul Pengujian	Prosedur Pengujian	Masukan	Keluaran	Kesimpulan
Registrasi Reseller	• Pilih Form Registrasi Reseller • Isi data registrasi yang diminta • Klik Kirim untuk mengirimkan data ke admin • Admin memverifikasi data registrasi	Isi data nama, alamat email, dan data lainnya Verifikasi oleh admin	Reseller telah berhasil register dan memiliki akun reseller	Berhasil
Login Reseller	• Pilih Menu Login Admin • Masukkan username “reseller@gmail.com” dan password “resell123” • Klik Login	Username “reseller@gmail.com” dan password “resell123	Reseller dapat masuk ke dalam sistem informasi dan memilih menu yang tersedia	Berhasil
Pemesanan Produk	• Buka katalog produk • Pilih produk yang diinginkan • Tentukan jumlah produk yang akan dipesan • Sistem menghitung total biaya dan membuat invoice	Reseller memilih: Produk A Jumlah produk: 10 unit Metode pembayaran: Transfer Bank	Pesanan reseller telah berhasil diproses	Berhasil

	• Pilih metode pembayaran • Lakukan pembayaran • Sistem memvalidasi pembayaran • Sistem memperbarui stok produk • Sistem mengirimkan konfirmasi pemesanan kepada reseller	Nominal yang dibayarkan: Rp500.000		
Pencatatan Laporan Penjualan	• Login akun reseller • Pilih Form Penjualan Reseller • Isi data penjualan yang diminta: - Produk yang terjual - Jumlah produk yang terjual - Harga jual per produk - Tanggal transaksi - Total pendapatan	Produk yang terjual: Produk A Jumlah produk yang terjual: 5 unit Harga jual per produk: Rp50.000 Tanggal transaksi: 2025-01-07 Total pendapatan: Rp250.000 (5 unit x Rp50.000)	Data penjualan berhasil dikirim dan laporan berhasil disimpan	Berhasil
Pengelolan Stok Produk	• Login ke akun admin • Buka dashboard stok produk • Admin menambah stok produk yang rendah • Klik perbarui data stok di sistem	Jumlah stok yang ingin ditambahkan: Produk D: 50 unit	Stok produk berhasil ditambahkan	Berhasil

A. Deployment
Deploy Aplikasi ke Streamlit
Sistem Informasi Kemitraan Reseller Perusahaan Footwear Harmon Corp

Repository Github
Github Harmon Corp

Implementasi Design Pattern

Design pattern adalah solusi standar untuk masalah-masalah umum dalam desain perangkat lunak, dirancang sebagai panduan atau kerangka kerja fleksibel yang dapat disesuaikan untuk mengatasi tantangan dalam penulisan kode [3].

A. Chain of Responsibility

Chain of Responsibility adalah pola desain perilaku yang memungkinkan objek untuk meneruskan permintaan (request) melalui serangkaian objek pengendali (handler). Setiap handler dalam rantai memutuskan apakah ia akan memproses permintaan tersebut atau meneruskannya ke handler berikutnya. Pola desain ini memungkinkan pemisahan tanggung jawab, meningkatkan fleksibilitas dalam menambah atau mengubah alur pemrosesan tanpa merubah kode di handler lain.

Dalam Sistem Informasi Kemitraan Reseller pada Perusahaan Footwear Harmon Corp, Chain of Responsibility digunakan dalam proses registrasi dan login pengguna. Setiap permintaan yang datang, seperti registrasi akun atau login, diproses melalui serangkaian handler yang melakukan validasi, pengecekan, dan akhirnya penyimpanan data (untuk registrasi).

Berikut adalah alur implementasi Chain of Responsibility pada sistem registrasi dan login:

1. Handler (Antarmuka):
   DEFINE Handler as an abstract class

   METHOD set_next(handler)

   METHOD handle(request)

   Kelas ini adalah antarmuka dasar yang mendefinisikan dua metode penting:
   set_next(handler): Digunakan untuk menetapkan handler berikutnya dalam alur.
   handle(request): Menangani permintaan dan meneruskan ke handler berikutnya jika ada.

2. AbstractHandler (Handler Umum):
   DEFINE AbstractHandler as a class implementing Handler

   PROPERTY _next_handler = None

   METHOD set_next(handler):

   SET _next_handler to handler

RETURN handler

METHOD handle(request):

IF _next_handler exists:

CALL _next_handler.handle(request)

RETURN None

Kelas ini mengimplementasikan antarmuka Handler dan menyimpan referensi ke handler berikutnya. Ini memiliki dua fungsi:

- set_next(handler): Menghubungkan handler berikutnya dalam rantai.
- handle(request): Jika ada handler berikutnya, maka permintaan akan diteruskan ke handler tersebut.

3. Handler 1: EmailValidationHandler
DEFINE EmailValidationHandler as a subclass of AbstractHandler
METHOD handle(request):
EXTRACT email from request
IF email is empty:
RETURN "Email tidak boleh kosong."
IF email format is invalid:
RETURN "Format email tidak valid."
RETURN super().handle(request)

Handler ini memverifikasi format email yang diterima dari permintaan.

- Jika email kosong atau tidak sesuai format yang valid, mengembalikan pesan kesalahan terkait validitas email.
- Jika valid, permintaan diteruskan ke handler berikutnya.

4. Handler 2: PasswordValidationHandler
DEFINE PasswordValidationHandler as a subclass of AbstractHandler
METHOD handle(request):
EXTRACT password from request
IF password is empty:
RETURN "Password tidak boleh kosong."
IF password doesn't meet criteria (length < 8, missing uppercase, lowercase, or number):
- RETURN "Password harus mengandung setidaknya 8 karakter, satu huruf besar, satu huruf kecil, dan satu angka."
- RETURN super().handle(request)
Handle ini untuk memverifikasi kekuatan dan kelayakan password.

- Jika password kosong atau tidak memenuhi kriteria tertentu (termasuk panjang minimum, huruf besar, huruf kecil, dan angka), mengembalikan pesan kesalahan.
- Jika valid, permintaan diteruskan ke handler berikutnya.

5. Handler 3: ConfirmPasswordHandler
DEFINE ConfirmPasswordHandler as a subclass of AbstractHandler

METHOD handle(request):
IF confirm_password is empty:
RETURN "Konfirmasi password tidak boleh kosong."
IF password doesn't match confirm_password:
RETURN "Konfirmasi password tidak sesuai."
RETURN super().handle(request)

Handler ini untuk memeriksa kecocokan antara password dan konfirmasi password.

- Jika konfirmasi password kosong atau tidak cocok dengan password yang diberikan, mengembalikan pesan kesalahan.
- Jika cocok, permintaan diteruskan ke handler berikutnya.

6. Handler 4: EmailExistsHandler

DEFINE EmailExistsHandler as a subclass of AbstractHandler
METHOD handle(request):
EXTRACT email from request
IF email exists in CSV file:
RETURN "Email sudah terdaftar."
RETURN super().handle(request)

Handler ini untuk memeriksa apakah email sudah terdaftar dalam database (file CSV).

- Jika email sudah terdaftar, mengembalikan pesan kesalahan bahwa email sudah digunakan.
- Jika tidak terdaftar, permintaan diteruskan ke handler berikutnya.

7. Handler 5: SaveUserHandler

DEFINE SaveUserHandler as a subclass of AbstractHandler
METHOD handle(request):
FOR each required field (name, address, contact):
IF the field is empty:
RETURN "<field> tidak boleh kosong."
STORE user data in CSV file
RETURN "Registrasi berhasil."

Handler ini untuk menyimpan data pengguna setelah semua validasi berhasil.

- Memeriksa apakah semua kolom wajib (nama, alamat, dan kontak) sudah terisi.
- Jika semua validasi berhasil, data pengguna disimpan dalam file CSV dan mengembalikan pesan bahwa registrasi berhasil.

8. Handler 6: VerificationStatusHandler

DEFINE VerificationStatusHandler as a subclass of AbstractHandler
METHOD handle(request):
EXTRACT email from request
IF email is found in CSV and status is not verified:
RETURN "Akun Anda belum diverifikasi oleh admin."
RETURN super().handle(request)

Handler ini untuk memeriksa status verifikasi akun pengguna.

- Jika akun ada tetapi statusnya belum diverifikasi, mengembalikan pesan bahwa akun belum diverifikasi oleh admin.
- Jika akun terverifikasi atau tidak ada masalah, permintaan diteruskan ke handler berikutnya.

Setiap handler di atas menangani bagian tertentu dari data permintaan. Begitu pengguna mengirimkan data registrasi, data tersebut diproses oleh setiap handler. Jika ada kesalahan di salah satu handler, alur akan dihentikan dan kesalahan akan dikembalikan. Jika semua validasi berhasil, data akan disimpan dan proses registrasi selesai.

B. Template Method

Template Method adalah pola desain perilaku yang mendefinisikan kerangka algoritma dalam kelas induk (superclass), namun memberikan fleksibilitas bagi kelas turunan (subclass) untuk mengganti langkah-langkah tertentu dalam algoritma tersebut tanpa mengubah keseluruhan struktur atau urutan proses. Dengan menggunakan Template Method, kelas induk menetapkan urutan langkah-langkah yang harus diikuti, tetapi kelas turunan dapat memodifikasi atau mengimplementasikan sebagian dari langkah-langkah tersebut.

1. DEFINE Abstract Class: StockManagement
   CLASS StockManagement (ABC):
   METHOD manage_stock(item=None, quantity=None, price=None):
   CALL view_stock()
   IF can_add_stock():
   IF item AND quantity AND price:
   CALL add_stock(item, quantity, price)
   IF can_edit_stock():
   CALL edit_stock()
   IF can_delete_stock():
   CALL delete_stock()

   METHOD load_data():
   RETURN data from "data_stock.csv"

   METHOD save_data(data):
   SAVE data to "data_stock.csv"

   METHOD view_stock():
   PRINT "Viewing Stock"
   RETURN data from load_data()

   ABSTRACT METHOD can_add_stock()
   ABSTRACT METHOD can_edit_stock()
   ABSTRACT METHOD can_delete_stock()

METHOD add_stock(item, quantity, price):
LOAD data from "data_stock.csv"
CREATE new data for item, quantity, price
SAVE new data to "data_stock.csv"
RETURN "Produk berhasil ditambahkan"

METHOD edit_stock(product_name, updated_quantity, updated_price):
LOAD data from "data_stock.csv"
UPDATE data for product_name with new quantity and price
SAVE updated data to "data_stock.csv"
RETURN "Produk berhasil diperbarui"

METHOD delete_stock(product_name):
LOAD data from "data_stock.csv"
REMOVE product_name from data
SAVE updated data to "data_stock.csv"
RETURN "Produk berhasil dihapus"

METHOD update_stock_after_payment(product_name, quantity):
LOAD data from "data_stock.csv"
DECREASE quantity for product_name
SAVE updated data to "data_stock.csv"
PRINT "Stok telah diperbarui."

StockManagement adalah kelas abstrak yang mendefinisikan langkah-langkah dasar dalam proses manajemen stok. Kelas ini mengatur operasi dasar terkait dengan stok produk, termasuk memuat data stok. Kelas ini juga mengimplementasikan metode Template Method yang mendefinisikan urutan langkah-langkah yang harus diikuti dalam proses manajemen stok. Template method ini menggabungkan metode umum yang digunakan untuk menangani stok produk, tetapi menempatkan keputusan tentang apakah langkah-langkah tertentu bisa dilakukan (seperti menambah, mengedit, atau menghapus stok) ke dalam metode abstrak yang harus diimplementasikan oleh subkelas.

Metode manage_stock mengelola urutan langkah-langkah ini dengan menyesuaikan akses yang dimiliki oleh subkelas yang mengimplementasikannya. Misalnya, reseller tidak dapat menambah atau mengedit stok, sedangkan admin dapat melakukannya.

2. DEFINE ResellerStockManagement as subclass of StockManagement
   CLASS ResellerStockManagement (StockManagement):
   CONSTRUCTOR:
   SET cart = []

```
        SET transactions_file = "reseller_products.csv"

        METHOD can_add_stock():
        RETURN False

        METHOD can_edit_stock():
        RETURN False

        METHOD can_delete_stock():
        RETURN False

        METHOD add_to_cart(product_name, quantity, total_harga):
        LOAD price from "data_stock.csv"
        CALCULATE total_harga = price * quantity
        ADD product details to cart

        METHOD get_cart_details():
        RETURN cart as DataFrame

        METHOD calculate_total_amount():
        CALCULATE total amount from cart
        RETURN total amount

        METHOD select_payment_method(payment_method, total_amount):
        CALL payment_method.pay(total_amount)
        FOR each item in cart:
        CALL update_stock_after_payment(item["product_name"],
item["quantity"])
        CALL record_transaction()

        METHOD update_stock_after_payment(product_name, quantity):
        LOAD data from "data_stock.csv"
        DECREASE quantity for product_name
        SAVE updated data to "data_stock.csv"
        RETURN "Stok berhasil diperbarui."

        METHOD record_transaction():
        LOAD cart data as transaction
        TRY:
        LOAD existing transaction data
        COMBINE new transaction with existing data
        SAVE to "reseller_products.csv"
        EXCEPT FileNotFoundError:
        SAVE new transaction to "reseller_products.csv"
```

PRINT "Transaksi berhasil dicatat."

ResellerStockManagement adalah subkelas dari StockManagement yang mengimplementasikan metode-metode dari kelas induk, tetapi membatasi hak akses tertentu sesuai dengan kebutuhan reseller. Dalam hal ini, reseller hanya dapat melihat stok dan menambahkan produk ke dalam keranjang mereka, tetapi tidak dapat menambah, mengedit, atau menghapus produk dari stok.

Metode manage_stock tidak digunakan secara langsung di ResellerStockManagement, karena reseller tidak memiliki hak akses untuk mengelola stok secara langsung. Sebagai gantinya, reseller dapat menambahkan produk ke keranjang menggunakan metode add_to_cart dan kemudian memilih metode pembayaran dengan select_payment_method.

3. DEFINE AdminStockManagement as subclass of StockManagement
   CLASS AdminStockManagement (StockManagement):
   METHOD can_add_stock():
   RETURN True

   METHOD can_edit_stock():
   RETURN True

   METHOD can_delete_stock():
   RETURN True

   METHOD get_total_expenditure():
   TRY:
   LOAD transaction data from "reseller_products.csv"
   CALCULATE total expenditure from "total_harga"
   RETURN total expenditure
   EXCEPT FileNotFoundError:
   RETURN 0

   METHOD view_transactions():
   TRY:
   LOAD transaction data from "reseller_products.csv"
   RETURN transaction data
   EXCEPT FileNotFoundError:
   RETURN empty DataFrame

AdminStockManagement adalah subkelas dari StockManagement yang memiliki kontrol penuh atas manajemen stok. Admin memiliki hak akses untuk menambah, mengedit, dan menghapus stok sesuai dengan kebutuhan. Selain itu, admin juga dapat melihat transaksi dan menghitung pengeluaran total dari transaksi reseller yang ada. Sebagai contoh, admin dapat melihat total pengeluaran menggunakan metode get_total_expenditure, yang mengakses file transaksi untuk menghitung total

pembelian dari reseller. Admin juga dapat melihat riwayat transaksi yang dilakukan oleh reseller dengan menggunakan metode view_transactions.

Dengan implementasi Template Method ini, kode menjadi lebih fleksibel dan terstruktur, memisahkan logika yang dapat digunakan bersama dan yang perlu dikustomisasi, sekaligus mengurangi duplikasi kode di subkelas.

C. Strategy

Strategy adalah pola desain perilaku yang memungkinkan Anda untuk mendefinisikan keluarga algoritma, menempatkan masing-masing algoritma ke dalam kelas yang terpisah, dan membuat objek-objeknya dapat dipertukarkan. Pola desain ini memungkinkan Anda untuk memilih algoritma tertentu di waktu runtime tanpa mengubah kode klien yang menggunakan algoritma tersebut.

Pada Sistem Informasi Kemitraan Reseller pada Perusahaan Footwear Harmon Corp, pola Strategy diimplementasikan untuk menangani berbagai metode pembayaran yang dapat dipilih oleh reseller saat melakukan pemesanan produk. Dengan menggunakan strategi pembayaran yang terpisah, sistem memungkinkan untuk memperluas dan menambahkan metode pembayaran baru dengan mudah tanpa mengubah struktur dasar kode yang ada. Berikut adalah alur bagaimana Strategy diterapkan dalam alur pemesanan:

1. DEFINE PaymentStrategy as an abstract class
   METHOD pay(total_amount) // Abstract method for making payment
   Ini adalah kelas abstrak yang mendefinisikan antarmuka umum untuk semua strategi pembayaran. Kelas ini memiliki metode pay yang bersifat abstrak, yang harus diimplementasikan oleh semua subclass. Metode ini bertanggung jawab untuk menangani proses pembayaran sesuai dengan strategi yang dipilih.
2. DEFINE CreditCardPayment as a subclass of PaymentStrategy
   METHOD pay(total_amount)
   RETURN "Pembayaran menggunakan Kartu Kredit sebesar Rp{total_amount} berhasil."
   Kelas ini adalah salah satu implementasi konkret dari PaymentStrategy. Kelas ini menangani pembayaran menggunakan Kartu Kredit. Metode pay di sini mengembalikan pesan sukses yang menunjukkan bahwa pembayaran telah berhasil dilakukan dengan kartu kredit.
3. DEFINE DigitalWalletPayment as a subclass of PaymentStrategy
   METHOD pay(total_amount)
   RETURN "Pembayaran menggunakan Dompet Digital sebesar Rp{total_amount} berhasil."
   Ini adalah implementasi lain dari PaymentStrategy, yang digunakan untuk pembayaran dengan Dompet Digital. Metode pay mengembalikan pesan sukses yang sesuai dengan metode pembayaran ini.
4. DEFINE BankTransferPayment as a subclass of PaymentStrategy
   METHOD pay(total_amount)

RETURN "Pembayaran melalui Transfer Bank sebesar Rp{total_amount} berhasil."

Kelas ini adalah implementasi konkret dari PaymentStrategy untuk menangani pembayaran melalui Transfer Bank. Metode pay mengembalikan pesan sukses yang menegaskan bahwa pembayaran melalui transfer bank telah berhasil.

5. DEFINE PAYMENT_METHODS as a dictionary

   KEY: "Kartu Kredit" --> Value: CreditCardPayment

   KEY: "Dompet Digital" --> Value: DigitalWalletPayment

   KEY: "Transfer Bank" --> Value: BankTransferPayment

   PAYMENT_METHODS adalah sebuah peta (dictionary) yang menghubungkan nama metode pembayaran (seperti "Kartu Kredit", "Dompet Digital", atau "Transfer Bank") dengan kelas strategi yang sesuai. Ini memungkinkan pemilihan strategi yang sesuai secara dinamis berdasarkan input atau kebutuhan pengguna. Misalnya, jika pengguna memilih "Kartu Kredit", aplikasi dapat secara otomatis menggunakan CreditCardPayment untuk menangani pembayaran.

Pola Strategy dalam proyek Sistem Informasi Kemitraan Reseller untuk mendefinisikan berbagai metode pembayaran dalam kelas terpisah dan memilih strategi pembayaran yang sesuai dengan kebutuhan pemesanan di waktu runtime. Dengan pendekatan ini, kita dapat dengan mudah menambah, mengubah, atau mengganti algoritma pembayaran tanpa mempengaruhi bagian lain dari sistem, menjadikan sistem lebih fleksibel dan mudah dikembangkan.

REFERENSI

[1] M. Alda, “Rancang Bangun Sistem Informasi Company Profile Berbasis Android Menggunakan Metode Extreme Programming (XP),” *JIKO (Jurnal Informatika dan Komputer)*, vol. 7, no. 1, p. 62, Feb. 2023, doi: 10.26798/jiko.v7i1.701.

[2] A. Saputro, D. Aly Syabibi, R. Aditya Nugraha, A. Andhyka, and S. Mu, “Implemetasi metode WaterFall Pada Sistem Informasi Inventori Perdana Cellular Group,” *Jurnal Sistem Informasi Dan Informatika-JISKA*, vol. 1, no. 2, p. 60, 2023, [Online]. Available: http://jurnal.unidha.ac.id/index.php/jteksis

[3] Refactoring Guru, “Design Pattern.” Accessed: Jan. 07, 2025. [Online]. Available: https://refactoring.guru/design-patterns